Detecting Clean Architecture Violations using Dynamic Context and LLMs: a Java Spring Case Study

Author: Aziz Vundirov

Supervisor: Andrey Sadovykh

November 1, 2026

Chapter 1

Introduction

1.1 Background

Software architecture decides how expensive a system will be to change. Clean Architecture [1] addresses that cost by arranging a system into concentric layers — entities, use cases, interface adapters, and frameworks and drivers — and by imposing the *Dependency Rule*: source-code dependencies may point only inwards, so that business rules never depend on the user interface, the database or the framework. Where control has to flow outwards, the dependency inversion principle is applied: the inner layer declares an interface, and the outer layer implements it. The value of the arrangement is realised only if it survives maintenance, and it usually does not. Architectural degradation is a recognised and extensively measured phenomenon: a multivocal review of 108 academic and grey sources catalogues 54 metrics and 31 measurement techniques for detecting it [2]. Conformance to the intended architecture therefore has to be re-checked continuously rather than established once.

Three families of tools do this check today, and all three are well studied. *Ar-*

chitecture compliance checking tools verify the implemented dependency graph against a declared module model; a benchmark of ten such tools reported an average sensitivity of 77% over 34 common dependency types in purpose-built test code and 72% over 109 hand-catalogued dependencies in a class of FreeMind, and identified ten dependency types that are systematically hard to detect [3]. *Architecture tests*, of which ArchUnit [4] is the Java representative, let a developer state rules such as "no class in the domain package may depend on the persistence package" as ordinary unit tests that a build tool can run; in a study that applied them to 26 student team projects, the largest part of the authors' effort went into tailoring those tests by hand to the package naming each team had chosen [5]. *General static analysers* such as SonarQube report code smells, bugs and vulnerabilities against a built-in rule set, but have no model of the intended architecture at all [6]. What unites the three families is that every rule has to be written and maintained by a human expert, and that only properties expressible as a dependency or a naming constraint can be checked at all.

Large language models change the second of these constraints: they can be asked about intent rather than about imports, so a violation that no dependency rule can express — a domain service that formats HTML e-mail, say — becomes checkable in principle. They soften the first without removing it, since a rubric still has to be written by hand, but it is written once and is not tied to the package names of one project. Recent work shows that such models can be made precise enough to be trusted. Gonzalez Bravo et al. [7] bind the model with three artefacts: a rubric of eight Clean Architecture criteria (CA01–CA08), a package of *traceable static evidence* per module, extracted from source structures, imports, framework references, persistence artefacts, tests and module organisation, and a strict JSON output contract that forces the model to cite the evidence before issuing a verdict.

From an initial corpus of 50 open-source enterprise repositories, 12 repositories and 36 modules were selected; two hosted model configurations produced 1136 auditable candidate findings; three experts reviewed every row blindly, and all 1136 were consensus-supported, with a mean absolute score error of 0.0118 and a severity agreement of 0.9525. Rubric-constrained, evidence-based LLM assessment of Clean Architecture is thus established as feasible and auditable.

1.2 Problem statement

What remains unknown is what such a pipeline cannot see. Its evidence is extracted from repository structure and source text, and the systems are neither built nor run. For a framework such as Spring, this is precisely where the architecture lives: dependency injection and aspect-oriented programming are realised through configuration files, annotations, reflection and dynamically generated proxies, so the binding between a port and its adapter does not appear in the source at all. The effect is measurable in the data of [7]. Of the eight rubric criteria, CA05, dependency inversion, has the lowest evidence-sufficiency rate, 0.6620, against 0.8028–1.0000 for the other seven, and the lowest severity agreement, 0.8873, against 0.9437–1.0000 for the rest. The authors attribute this to interfaces, injection mechanisms and adapter boundaries that are “implicit, configured externally or framework-dependent”, name the static-evidence design as a threat to internal validity, and leave stronger extraction of runtime wiring evidence as future work.

A second gap concerns measurement rather than evidence. The gold standard of [7] is assembled from the findings the model itself produced and then validated by experts, which establishes precision but says nothing about what the

pipeline missed; the word *recall* does not occur anywhere in the article, and false negatives are never counted. The same asymmetry runs through the LLM-based conformance literature: reported numbers describe the quality of what was found, because no benchmark of Clean Architecture violations with a known, complete ground truth exists for this task.

The evidence that would close the first gap already exists, but in a different research community. Jasmine [8] extends Soot's intermediate representation [9] with explicit models of Spring's core technologies: it parses XML configuration and annotations, synthesises a factory class that materialises singleton and prototype beans, and emits proxy classes that attach AOP advice in the correct order, so that a downstream analysis sees the resolved call graph. The gain is substantial — the completeness of reachable application methods rises by up to 28.14% over the state-of-the-art Spring-aware baseline, while time and memory grow by less than 10% on most of the benchmark projects — and when the enriched graph was handed to FlowDroid [10], twelve information-leakage paths were confirmed by hand and reported to the maintainers of widely used repositories. What matters for Clean Architecture is not the extra reachability as such but one of its by-products: resolving an `@Autowired` field names the concrete class the container actually injects, which is exactly the fact CA05 needs and static evidence cannot supply. Framework-aware dynamic context and LLM-based Clean Architecture assessment, however, have not been brought together. Jasmine has been used for taint analysis and vulnerability detection; a citation check over the works citing and cited by it (Section 2.1) found none that applies it to architecture conformance, and none in which an evidence-based LLM pipeline consumes a resolved Spring call graph.

1.3 Research question and hypothesis

This thesis asks whether that missing intersection accounts for the weakness reported above. The research question is: *to what extent does enriching an evidence-based LLM conformance pipeline with dynamic context — resolved dependency injection bindings, AOP advice attachments and a Spring-aware call graph — change the number of Clean Architecture violations detected in Java Spring modules, and for which rubric criteria is the change largest?* A secondary question asks what this costs in precision, since adding context is not free: on SWE-bench, resolve rates fall from 1.96% to 1.22% as the retrieved context grows from 13k to 50k tokens [11], and tightening structural constraints on a generation task degrades it further [12].

The hypotheses follow the diagnosis of [7] directly.

H0. Enriching the static evidence package with dynamic context extracted from a Jasmine-augmented Spring call graph leaves the recall of the LLM pipeline unchanged on every rubric criterion.

H1. That enrichment increases recall on the criteria that depend on runtime wiring, and the increase is largest for CA05, dependency inversion.

H1 is the directional alternative to H0 and is the hypothesis this thesis sets out to test; rejecting H0 without the per-criterion pattern predicted by H1 would be a weaker, but still informative, outcome.

1.4 Approach

To test this, the pipeline of [7] is reimplemented for Java and extended by one stage. A module is compiled, Soot builds the base call graph [9], Jasmine resolves Spring's dynamic features [8], an extractor turns both the static and the resolved facts into a JSON evidence package per rubric criterion, and the LLM issues findings under the same strict output contract. The experiment is an ablation: the identical rubric, prompt, models and output schema are run with and without the Jasmine stage, so that the measured difference is attributable to dynamic context alone.

Because recall cannot be computed without a complete ground truth, the benchmark is constructed rather than sampled. Clean Spring reference projects are mutated by a catalogue of formal operators, one or more per rubric criterion, each recorded in a manifest that fixes the injected violation, its location and the criterion it breaks. Deliberate mutation of a conforming artefact is taken from Rubei et al. [13], who manually mutated one reference architecture and four software architectures into 16 faulty variants; the step from their manual mutation to a catalogue of reusable operators is a contribution of this thesis. Unmutated control cases, which must raise no finding, follow the paired design of the ACC benchmark of Pruijt et al. [3]. The decisive operator class is *pseudo-inversion*: an interface is kept in place, but its single implementation is moved into the same inner package, so that the code still looks inverted while the resolved binding shows a direct dependency. Static evidence cannot separate this from a genuine port; a resolved `@Autowired` binding can. Scope is deliberately narrow — Java only, Spring and Spring Boot with annotation-based and XML configuration, module-level analysis as in [7].

The expected outcome is a per-criterion recall difference that is concentrated on the runtime-dependent criteria and close to zero elsewhere, together with the first precision and recall figures for this task measured against a known ground truth rather than against a model's own output. A negative result is equally publishable: if the enriched evidence does not raise recall, or raises it at the cost of precision, this bounds how much of the CA05 weakness is a matter of evidence at all, as opposed to a limit of the rubric or of the models.

1.5 Significance

Three consequences follow. For architecture conformance checking, the work tests whether the long-standing blind spot of static analysis — dependency injection, reflection and dynamic wiring, which the tools surveyed in Chapter 2 either name as a limitation or leave outside their scope altogether — can be closed by feeding a framework-aware call graph to a model instead of by writing more rules by hand. For the evaluation of LLM-based architecture tools, it supplies a reusable mutation-seeded benchmark on which recall, not only expert-validated precision, can be reported. And for practice, a detector whose false positives stay low is the precondition for anything further: across 421 976 SonarQube issues from 246 projects the average resolution rate per project was 13%, and in aggregate only 8.76% were ever fixed [6], so a detector that merely adds to that pile helps nobody. Repairing what is detected is a separate problem and is left outside the scope of this thesis: the multivocal review of architectural degradation finds that tools detect decay but rarely support ongoing or preventive remediation [2], and in a dedicated benchmark of architectural smell repair the best of 11 agent configurations resolved 47.7% of the cases while the most aggressive introduced 140

new smells [14].

Chapter 2

Literature Review

2.1 Scope and method of the review

The review answers four questions, which also give the chapter its structure: what Clean Architecture is and how conformance to it is made measurable (Section 2.2); how conformance is checked today by rule-based means, and what those means cannot see (Section 2.3); what is known about using language models for this class of task, and how the result depends on what is put into the model (Sections 2.4 and 2.5); and whether the missing evidence — the runtime wiring of a framework — can be recovered by analysis (Section 2.6). Section 2.7 examines how the field establishes ground truth, and Section 2.8 states the resulting gap.

The works reviewed here were collected by keyword search over *clean architecture*, *architecture conformance*, *architectural erosion*, *large language model*, *ArchUnit*, *SonarQube*, *dependency injection* and *Spring* A source was retained if it either evaluated a technique for detecting architectural or design violations, or reported empirical evidence on how context affects a language model on a software-engineering task. Work on automatic *repair* of violations is reviewed only to de-

limit the scope (Section 2.8). This chapter makes several claims that no study has yet done a particular thing. Such claims cannot be established by keyword search, so they were checked separately: on 1 November 2026 the works citing, and the works cited by, the two studies closest to this thesis [7], [8] were enumerated through the Semantic Scholar Graph API and screened by title, venue and abstract. The purpose of that pass was to test the negative claims of Sections 2.6 and 2.8, not to extend the corpus reviewed here; its scope and results are reported where those claims are made. **[проверить: в обзор пока не внесены работы, найденные этим проходом — Sentinel, FlowChecker, Micans, Perry & Wolf, Murphy reflexion models и остальные из snowball-log.md]**

2.2 Clean Architecture and its operationalisation

Clean Architecture [1] is used throughout as the single normative source. A system is divided into concentric layers — entities, use cases, interface adapters, frameworks and drivers — and governed by the Dependency Rule: source-code dependencies point inwards only. Where control must cross a boundary outwards, dependency inversion applies, so the inner layer owns an interface that the outer layer implements. The purpose of the arrangement is economic: it keeps the cost of change low during maintenance, which is where most of a system's lifetime cost is spent. Code-level defects are distinguished from architectural ones in the sense of *Clean Code* [15]: a code smell is local to a class or a method, whereas an architectural violation is a property of the relation between components.

The principles as stated in the book are prose, not predicates, and every tool reviewed below has to operationalise them before it can check anything. Two operationalisations recur. Rule-based tools reduce the principles to constraints over a

dependency graph, which is exact but narrow. The rubric of Gonzalez Bravo et al. [7] instead decomposes them into eight criteria, each paired with the observable indicators that signal it: CA01 dependency rule between layers, CA02 separation of domain, application, infrastructure and interface concerns, CA03 framework independence, CA04 database independence, CA05 dependency inversion, CA06 single responsibility and modular cohesion, CA07 testability, and CA08 evidence traceability. For CA05, for instance, the listed indicators are interfaces, services, dependency injection, adapters, ports and factories. This thesis adopts that rubric because it is scored criterion by criterion and was validated against expert judgement, which is what makes a per-criterion comparison possible; no claim is made here that it is the only such operationalisation, as no search was run that could establish that.

Independent evidence confirms that the principles erode in practice. A multivocal review of 108 academic and grey sources on architectural degradation identifies 54 metrics and 31 measurement techniques, and concludes that the available tools detect decay but rarely support continuous or preventive remediation [2]. Detection is thus the better-served half of the problem in terms of the number of available instruments, which is why this thesis asks not whether more detectors are needed but what the existing ones can and cannot see; repair, the half the review finds neglected, is outside its scope (Section 2.8).

2.3 Rule-based conformance checking

2.3.1 Static architecture compliance checking

Static architecture compliance checking (ACC) verifies an implemented dependency graph against a declared module model. Pruijt et al. [3] measured how accurately ten ACC tools actually perform that extraction, using two tests. The Benchmark test is purpose-built Java code covering 34 common dependency types (25 direct and 9 indirect), implemented as 64 test cases aimed at true positives and false negatives, mirrored by 64 control cases whose code is identical but whose from-classes sit in a sibling package, so that any violation reported against them is by construction a false positive. The FreeMind test applies a single rule to the class `ScriptingEngine`, in which 109 dependency-causing code constructs had been catalogued by hand and cross-checked by a second author; only five of the ten tools produced messages traceable enough to be scored at that level.

The results set a realistic ceiling for the whole family. Average sensitivity was 77% on the Benchmark test and 72% on the FreeMind test, where individual tools ranged from 58% to 83%. Ten dependency types proved hard to detect for several tools, and a frequency analysis over five open-source systems shows these account for at least 47% of the dependencies in real code. The authors distil four challenges: inheritance structures, where 43% of calls to inherited instance methods went undetected on average and no tool reported indirect inheritance dependencies at all; inner classes; standalone object references; and information that the compiler removes, which makes imports, uninitialised local declarations and constant-variable accesses invisible to tools that read only bytecode.

Two things follow for this thesis. First, ACC accuracy is bounded by the

accuracy of dependency extraction, and the bound is not tight even in the best case. Second, and more important for the argument developed here, the benchmark deliberately restricts itself to *common* structural constructs of the language; reflection, externally configured wiring and container-managed injection are not among the 34 types and are never discussed in the paper. The 77% ceiling is therefore measured on dependencies that are present in the source text. Dependencies that are not — the subject of Section 2.6 — fall outside what this literature even attempts to measure. The paired sensitivity and false-positive design is also the methodological precedent for the ground truth of this thesis (Section 2.7).

2.3.2 Architecture tests

ArchUnit [4] imports compiled classes, builds their dependency graph, and lets a developer express architectural rules as ordinary unit tests, so that a violation fails the build. De Luca et al. [5] used it to quantify how often a prescribed architecture is actually followed. They defined a four-layer reference architecture for student projects — GUI, Controller, Model and DAO — enumerated the eight couplings it forbids, and wrote an ArchUnit test for each, running them together with a SonarQube instance in a Docker container through each project's Maven build. The subjects were 26 projects written by 50 students, averaging 41 classes and 4 647 non-commenting lines of code.

The violation rates are high. Dependencies from DAO to Model appeared in 96% of the projects (25 of 26), from GUI to Model in 88% (23 of 26), and direct data access from the GUI to the DAO layer in 31% (8 of 26); the remaining five forbidden couplings occurred in at most one project each. Two caveats matter for how this study should be cited. It is a *post hoc* evaluation: the students were not

given the ArchUnit rules during development, so the result shows how often an unenforced architecture erodes, not that developers ignore available rules. And the authors report that the largest part of their effort went into manually tailoring the ArchUnit tests to the package naming each team had chosen — direct evidence for the maintenance cost that makes rule-based checking expensive to scale.

SecArchUnit [16] extends the same mechanism to security. Seven architectural security constraints were formalised — log all security events; enforce authentication and authorisation at a single point; send messages from a central point; validate user input; restrict thread spawning; keep sensitive information inside a trust boundary; and keep secrets out of log messages — and evaluated on three systems: JPetStore, ATM Simulation and iTrust. The comparison with SonarQube and PMD is more nuanced than it is often reported. Constraints 1–5 could be implemented in all three tools, and SonarQube in particular matched SecArchUnit closely on precision and recall. The difference is one of expressiveness: constraints 6 and 7 track the flow of information through the system, which neither SonarQube nor PMD inherently supports, so only SecArchUnit could evaluate them at all. On effort, excluding the first two constraints, SecArchUnit required 10 hours of implementation against 16 for SonarQube and 21 for PMD.

2.3.3 Conformance checking under dynamic language features

ArchRuby [17] is the clearest statement of the problem this thesis attacks, in a different language. Checking conformance in Ruby is hard precisely because dynamic invocations and dynamic object construction keep the dependency graph from being statically determined, so the tool combines static analysis with a lightweight type propagation heuristic and a domain-specific language in YAML

in which the architect declares the permitted module dependencies. Evaluated on three real-world systems with their own architects, it identified 48 architectural violations of which the developers had no prior knowledge; the type propagation heuristic raises the number of analysed types by 5% on average and by up to 17%, and the authors report that certain violations are detected only because of it.

Two limitations are stated by the authors and are directly relevant here. The rules must be written by the software architect of the system, who is trained on the specification language before the study begins; and the architects repeatedly had to refine those rules to eliminate false positives, to the point that the authors conclude that rule specification and conformance checking are in practice performed jointly rather than as separate phases. The broader argument transfers from dynamic typing to dynamic framework wiring: where the binding is decided outside the source text, a purely syntactic analysis cannot resolve it, and a heuristic that recovers part of it changes what can be detected.

2.3.4 General static analysis

SonarQube is the most widely deployed static analyser, but it has no model of the intended architecture; it reports code smells, bugs and vulnerabilities with severities from Blocker to Info. Marcilio et al. [6] studied its realistic usage rather than its behaviour on software revisions, mining 421 976 issues from 246 projects across four SonarQube instances: two in open-source communities (Eclipse and Apache) and two in Brazilian government institutions. The tool is not the problem. In an accompanying survey answered in full by 18 developers from those organisations, more than 80% agreed or strongly agreed that the issues reported by static analysis tools are relevant for improving the design and implementation

of software; the authors themselves describe what follows as an apparent contradiction.

What fails is the disposal of the findings. The average resolution rate per project was 13%, and in aggregate only 36 974 issues — 8.76% of the total — were ever fixed. Of those that were fixed, the median delay was 18.99 days, which the authors note is shorter than previously reported periods for bugs; but 29% of the fixes took more than a year, and almost half of the issues still open had been reported more than a year earlier. The composition of the fixes is equally telling: code smells account for 79.6% of everything fixed, against 5.8% bugs and 3.9% vulnerabilities, and by severity 68% of the fixes are Major while Minor accounts for a further 21%. Severity does influence how fast an issue is fixed ($p < 2.2 \times 10^{-16}$), but not in the direction one would expect: Blocker and Minor issues are closed faster than Critical and Major ones, and the authors call the speed of Minor fixes surprising, conjecturing that they are simply the easiest.

The authors' own interpretation is the one that matters here: they conjecture that only a subset of the checkers reveal real design and coding flaws, and that the rest artificially inflate the measured technical debt of a system. An unprioritised, unexplained issue list therefore degrades into a backlog that obscures rather than reveals technical debt. Any detector proposed as a replacement has to be judged on false positives and on the legibility of its output, not on raw detection counts alone.

2.4 Language models on software-engineering tasks: the role of context

Before reviewing LLM-based conformance checking, it is necessary to establish what is known about the failure modes of these models on code, because the design of the pipeline and of the evaluation in Chapter **??** responds to them.

The first is context dilution. SWE-bench [11] gives a model a real repository and an issue description and requires a patch that makes the failing tests pass without breaking the passing ones. Across 2 294 task instances drawn from 12 Python repositories, the strongest model of the time, Claude 2, resolved 1.96% in the realistic BM25 retrieval setting against 4.8% when handed exactly the files the original developer touched. The decisive measurement for this thesis is what happens as the context grows: with the same retriever, the resolve rate falls from 1.96% at a 13k-token limit to 1.87% at 27k and 1.22% at 50k, even though the larger budgets cover more of the files actually needed. A code base cannot simply be pasted into a prompt; the evidence must be selected and structured, which is exactly what the evidence package of [7] does.

The second is constraint decay. When a backend-generation task is tightened by adding a layered architecture, a database and an ORM to a bare framework, the share of passed assertions falls monotonically: across the configurations that are capable at baseline, the mean drops from about 79% with no added constraint to about 50% with all three, an average loss of 27.28 points over the full study of 80 greenfield and 20 feature-implementation tasks on eight web frameworks [12]. Three findings matter here specifically. The database requirement drives most of the degradation (-19.3 percentage points) while the Clean Architecture requirement costs -9.1 and the ORM requirement almost nothing. End-to-end

success collapses even where assertions largely pass: the strongest configuration reached a 78.6% assertion rate but only 8.3% pass@1 on fully specified tasks. And agents cope with explicit micro-frameworks such as Flask far better than with convention-heavy ones such as FastAPI and Django, where behaviour is implicit. A framework whose wiring is invisible to the model is thus already known to be the hard case, which is an argument for supplying that wiring explicitly rather than hoping the model infers it.

The third is non-determinism in the output itself. In a Clean Architecture and CQRS case study, 60 prompts were each executed five times against GPT-4.1 under an identical template, giving 300 invocations [18]. Of these, 4.33% were not parseable as valid CSV and 10.33% were rejected by validation for failing at least one application-level contract. Stability was worse than correctness: only 56.67% of prompts produced identical raw responses across all five runs, and 76.67% produced the same validated structured record. The authors conclude that prompt engineering alone cannot guarantee reliable structured output and that parsing, validation and a user confirmation step are mandatory — which motivates the strict JSON contract and the repeated runs in the evaluation design of this thesis.

Finally, where LLMs have been compared directly with static tools on a related detection task — nine code smell types across 30 Java systems, with four models — the best results came from neither alone but from the hybrid, which gave the best F1 for 5 of the 9 smells, while producing additional false positives on the more complex smell categories [19]. The field's own evidence therefore points towards feeding a model the output of an analyser instead of raw code, and towards watching precision when doing so.

2.5 LLM-based detection of architectural violations

2.5.1 Evidence-based rubric assessment

Gonzalez Bravo et al. [7] is the direct predecessor of this thesis. The pipeline constrains the model with the eight-criterion rubric, a per-module package of traceable static evidence extracted from source structures, imports, framework references, persistence artefacts, tests and module organisation, and a strict JSON contract whose two-step verification requires the model to cite concrete evidence — a path, a symbol, an annotation — and map it to a rubric item before scoring. From an initial corpus of 50 open-source enterprise repositories, 12 repositories and 36 modules were selected by analytically stratified selection; two hosted model configurations, DeepSeek V4 Flash and Qwen 3.6 Plus, produced 1136 auditable candidate findings. Three experts reviewed every row blindly, generating 3408 review records, and all 1136 findings were consensus-supported, with a mean absolute score error of 0.0118 and a severity agreement of 0.9525; agreement was reported with pairwise weighted Cohen's κ, Gwet's AC1/AC2, Krippendorff's α and McNemar's exact test. Against a lightweight deterministic rule-based baseline over the same 288 module-criterion units, the LLM workflow reduced the mean absolute score error from 0.8819 to 0.0142 and raised severity agreement from 0.4097 to 0.9479, with McNemar tests significant at $p < 0.001$. The authors are explicit that this is a comparison against a simple deterministic reference, not a claim of superiority over mature architecture-conformance tools, and name stronger external baselines as future work.

Three limitations are decisive for this thesis, and the authors state all three themselves. First, the systems were never built or run, so runtime dependency in-

jection, dynamic imports, reflection and deployment-time configuration are outside the evidence; this is flagged as a threat to internal validity and as particularly relevant to CA05. Second, the criterion-level results quantify the consequence: CA05 has an evidence-sufficiency rate of 0.6620 against 0.8028–1.0000 for the other seven criteria, and a severity agreement of 0.8873 against 0.9437–1.0000, the lowest on both measures; the stated cause is that interfaces, injection mechanisms and adapter boundaries can be implicit, externally configured or framework-dependent. Third, the gold standard is built from the model's own findings, so it measures precision and expert-supported validity; recall is never reported and false negatives are never counted.

2.5.2 Architectural decision records

Su et al. [20] test whether LLMs can detect violations of architectural decisions, using 980 ADRs from 109 GitHub repositories. Their pipeline indexes code and decision text in a FAISS vector store and retrieves per-ADR context, then has one reasoning model, Marco-o1, screen for violations while three independent validators — Mistral-NeMo, Qwen3 and Llama-3.1 — review the verdict together with its reasoning chain. Inter-model agreement was substantial (Fleiss' $\kappa = 0.724$), and on a manually validated sample of 305 cases the two best models reached accuracies of 0.911 and 0.904 against 0.852 and 0.829 for the weaker two. The error analysis is the useful part for this thesis: of the 92 sampled cases in which at least one model predicted incorrectly, semantic or logical misinterpretation accounted for 44.57% and inability to infer implicit or missing context for 28.26%, while by subject the largest group of failures — 42.39% — consisted of infrastructure and deployment-specific decisions, precisely those whose evidence

is not present in the source code.

2.5.3 Generating conformance tests from natural language

Lucena et al. [21] attack the authoring cost rather than the detection step: instead of requiring someone to formalise rules, they translate informally stated architectural rules into executable design tests. GPT-4o generated 30 natural-language design rules over six projects, each implementing a different design pattern, and the rules were classified into three difficulty levels (14, 12 and 4 rules respectively). CodeStral then generated a JUnit test per rule against the Design Wizard library, with retrieval-augmented generation over the library's indexed documentation to compensate for the library being under-represented in training data. Of the 30 tests, 29 (96.67%) compiled, 19 (63.33%) passed all assertions, and 23 (76.67%) were judged to express the intended rule. The eleven assertion failures split into five rule mismatches, four cases of library misuse and two ordinary language errors, and the failures concentrated in difficulty levels 2 and 3 — the model handles simple rules well and complex ones considerably worse. The approach keeps the determinism of a test suite while removing the formalisation effort, but it inherits the expressiveness limit of the underlying library: a rule that cannot be stated over the static structure cannot be generated either.

2.5.4 Recommending resolutions

Rubei et al. [13] sit one step after detection, adding an LLM-based recommender to a model-driven continuous conformance tool so that it proposes resolutions for detected violations. Methodologically they are relevant here for how they obtained their subjects: one reference architecture and four software architectures

from the IoT domain were manually mutated into 16 faulty architectures, and two models were compared by success rate on them. The mutation is of architectural models rather than of source code, and it is manual rather than operator-driven, but the principle — deliberately corrupting a conforming artefact so that the expected finding is known in advance — is the mechanism this thesis adopts to make recall measurable.

2.6 Framework-aware analysis and dynamic context

Everything above either ignores runtime wiring or names it as a limitation. The analysis community has addressed it directly for Spring. Chen et al. [8] begin from the observation that Spring's core technologies — dependency injection and aspect-oriented programming — are implemented with configuration files, reflection and dynamic proxies, and classify their effect on static analysis into two kinds. Under *explicit impact*, the call exists in the source but the analyser cannot tell which object is injected, so an `@Autowired` field is treated as null. Under *implicit impact*, the calls and data flows are entirely inside the framework: AOP advice and CGLIB or JDK dynamic proxies have no counterpart in the source at all.

Jasmine resolves both at the level of Soot's Jimple intermediate representation [9]. It first identifies the framework usage, parsing XML configuration and annotations against a catalogue of matching rules for `@Service`, `@Controller`, `@Autowired`, `@Aspect`, `@Around` and the rest, and extracting singleton and prototype beans, controller entry points, and target-method-to-advice bindings. It then synthesises new IR that models those features: a factory class holding the beans with getters, injected into constructors; proxy classes that insert advice calls

in the correct order; a synthetic entry method that reaches all controllers; and stubs for reflection and for persistence mappers. The output is ordinary Jimple, which Soot, Doop and FlowDroid consume unchanged.

The reported effect is a materially more complete call graph. On a micro-benchmark Jasmine outperforms the compared call-graph construction methods on all eight measures, and on a production benchmark of 18 real Spring projects (21 modules) the completeness of reachable application methods rises by between 0.21% and 28.14% over the state-of-the-art Spring-aware baseline, while time and memory grow by less than 10% on most projects. Handed to FlowDroid [10], the enriched graph exposed 44 candidate information-leakage paths in nine projects; after deploying the projects and exercising the corresponding functions the authors excluded 32 paths as unreachable through control flow and manually confirmed the remaining 12, which were reported to the maintainers. Two caveats are stated by the authors and carried into this thesis: some added edges are spurious, and because no dynamic baseline for Spring reachability exists, precision and recall could not be computed on the real-world projects.

The gap in this line of work is its purpose rather than its availability. Jasmine was evaluated for taint analysis and vulnerability detection, and the citation check described in Section 2.1 shows what it has been used for since: of the works citing it as of November 2026, most apply it to security analysis — taint tracking, access-control and injection vulnerabilities, directed fuzzing — or to call-graph and pointer-analysis infrastructure. One is a rule-checking system that integrates Jasmine as a framework-aware backend in order to recover Spring-mediated control flow before checking developer-written temporal properties, which establishes that a rule checker can consume Jasmine's output. None of them checks architectural rules, and none supplies the recovered wiring to a language model. On the

other side, of the works Jasmine itself cites, exactly one concerns conformance of any kind, and that is conformance to API protocols rather than to an architecture.

2.7 Benchmarks and ground truth

How the field establishes ground truth explains why recall is missing. Three patterns appear. Expert validation of tool output [7], [17], [20] yields precision and expert agreement but, by construction, cannot count what the tool did not report. Manual inspection of a sample [8] verifies individual findings but is not exhaustive. Constructed benchmarks, in which the expected findings are fixed in advance, do support recall: Pruijt et al. [3] built purpose-made test software covering 34 known dependency types, paired every positive case with a control case that must raise no violation, and catalogued 109 dependency-causing constructs in a real class by hand, while Rubei et al. [13] manually mutated five architectures into 16 faulty variants. The third pattern is the only one that answers the question this thesis asks, and none of the works reviewed here applies it to Clean Architecture violations in Spring code.

Table 2.1 summarises the primary studies along the two dimensions that matter for the gap: what evidence the technique consumes, and where its rules come from.

2.8 Research gap

Table 2.2 positions the reviewed work on two axes: whether the rules are written by hand or supplied as a rubric to a model, and whether the evidence is purely static or includes resolved framework wiring. Three of the four cells are

TABLE 2.1
Primary studies by evidence consumed and origin of the rules.

Work	Venue	Evidence consumed	Rules from	Key limitation
Pruijt et al. [3]	SPE 2017	static dependency graph	manual module model	77%/72% sensitivity; dynamic wiring out of scope
De Luca et al. [5]	EASE 2024	compiled classes	hand-written tests	tests tailored per project by hand
SecArchUnit [16]	Thesis 2020	compiled classes	hand-written DSL	7 constraints formalised by hand
ArchRuby [17]	JOT 2016	graph + type heuristics	manual YAML spec	architect must author and refine rules
Marcilio et al. [6]	ICPC 2019	AST per file	built-in rule set	no architecture model; 13% of issues fixed
Gonzalez Bravo et al. [7]	Computers 2026	static evidence package	CA01–CA08 rubric	no runtime evidence; recall unknown
Su et al. [20]	arXiv 2026	retrieved code + ADR text	ADR documents	fails where evidence is not in code
Lucena et al. [21]	SBES 2025	static structure	informal NL rules	limited to statically expressible rules
Chen et al. [8]	ASE 2022	resolved DI/AOP call graph	not applicable	applied to taint analysis, not architecture

occupied.

TABLE 2.2
Positioning of the reviewed approaches; the lower-right cell is unoccupied.

	Static evidence only	**Framework-aware dynamic context**
Rule-based	ACC tools, ArchUnit, SecArchUnit, SonarQube [3], [5], [6], [16] — exact but manually maintained and blind to intent	ArchRuby [17] — type propagation for dynamic typing; Ruby only, rules still manual
LLM-based	Rubric and evidence packages, ADR checking, test generation [7], [20], [21] — judges intent; CA05 weakest, recall unmeasured	**no work found in this review**

The gap is therefore specific rather than general. Rubric-constrained, evidence-based LLM assessment of Clean Architecture works and is auditable [7]; framework-aware callgraph construction for Spring works and measurably improves completeness [8]; and the weakest criterion of the former is weak for exactly the reason the latter was built to address. Neither the works reviewed here nor those surfaced by the citation check of Section 2.1 combine the two. The recall of the former is unreported, and none of the reviewed work supplies a benchmark with complete ground truth against which it could be measured; the mutation-seeding methodology that would supply one has been demonstrated for architectural models [13] and for dependency benchmarks [3], but not for Clean Architecture violations in Spring code.

This thesis addresses that intersection: it supplies resolved dependency injection bindings, AOP advice attachments and a Spring-aware call graph as additional evidence to a rubric-constrained LLM pipeline, and measures the per-criterion effect against a mutation-seeded benchmark in which the ground truth

is known by construction. Repair of the detected violations stays out of scope. Agents that attempt it remain unreliable — across 11 agent configurations from four model families, the best resolved 47.7% of architectural smell cases and the most aggressive introduced 140 new smells [14] — and in any case a detector with measured recall and low false positives is the precondition for attempting repair at all.

Bibliography cited

[1] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017. [Online]. Available: https://dl.acm.org/doi/abs/10.5555/3175742.

[2] N. Ahmad, R. Su, M. Esposito, A. Janes, V. Lenarduzzi, and D. Taibi, *Architectural degradation: Definition, motivations, measurement and remediation approaches*, 2025. arXiv: 2507.14547. [Online]. Available: https://arxiv.org/abs/2507.14547.

[3] L. Pruijt, C. Köppe, J. M. van der Werf, and S. Brinkkemper, "The accuracy of dependency analysis in static architecture compliance checking," *Software: Practice and Experience*, vol. 47, no. 2, pp. 273–309, 2017. DOI: 10.1002/spe.2421.

[4] ArchUnit contributors, *ArchUnit: A Java architecture test library*, https://www.archunit.org, Accessed: 2026-11-01, 2025.

[5] M. De Luca, S. Di Meglio, A. R. Fasolino, L. L. L. Starace, and P. Tramontana, "Automatic assessment of architectural anti-patterns and code smells in student software projects," in *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering (EASE)*, ACM, 2024. DOI: 10.1145/3661167.3661290.

[6] D. Marcilio, R. Bonifácio, E. Monteiro, E. Canedo, W. Luz, and G. Pinto, "Are static analysis violations really fixed? A closer look at realistic usage of SonarQube," in *Proceedings of the 27th IEEE/ACM International Conference on Program Comprehension (ICPC)*, IEEE, 2019, pp. 209–219. DOI: 10.1109/ICPC.2019.00040.

[7] E. J. Gonzalez Bravo, G. Chavira Juarez, E. Alvarez Navarro, A. Montoto Gonzalez, and J. L. Diaz Juarez, "An evidence-based LLM framework for expert-validated clean architecture conformance assessment in enterprise software systems," *Computers*, vol. 15, no. 10, p. 657, 2026. DOI: 10.3390/computers15100657. [Online]. Available: https://www.mdpi.com/2073-431X/15/10/657.

[8] M. Chen, T. Tu, H. Zhang, Q. Wen, and W. Wang, "Jasmine: A static analysis framework for Spring core technologies," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, ACM, 2022, pp. 1–13. DOI: 10.1145/3551349.3556910. [Online]. Available: https://weihang-wang.github.io/papers/ASE2022-Jasmine.pdf.

[9] R. Vallée-Rai, P. Co, E. Gagnon, L. Hendren, P. Lam, and V. Sundaresan, "Soot: A Java bytecode optimization framework," in *CASCON First Decade High Impact Papers*, Originally published at CASCON 1999, ACM, 2010, pp. 214–224. DOI: 10.1145/1925805.1925818.

[10] S. Arzt et al., "FlowDroid: Precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for Android apps," in *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Imple-*

mentation (PLDI), ACM, 2014, pp. 259–269. DOI: 10.1145/2594291.2594299.

[11] C. E. Jimenez et al., *SWE-bench: Can language models resolve real-world GitHub issues?* 2023. arXiv: 2310.06770. [Online]. Available: https://arxiv.org/abs/2310.06770.

[12] F. Dente et al., *Constraint decay: The fragility of LLM agents in back-end code generation*, 2026. arXiv: 2605.06445. [Online]. Available: https://arxiv.org/abs/2605.06445.

[13] R. Rubei, A. Di Salle, and A. Bucaioni, "LLM-based recommender systems for violation resolutions in continuous architectural conformance," in *2025 IEEE 22nd International Conference on Software Architecture Companion (ICSA-C)*, IEEE, 2025, pp. 404–409. DOI: 10.1109/ICSA-C65153.2025.00063.

[14] I. G. Dinu, M. C. Mihăescu, and T. Rebedea, *SmellBench: Evaluating LLM agents on architectural code smell repair*, 2026. arXiv: 2605.07001. [Online]. Available: https://arxiv.org/abs/2605.07001.

[15] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008.

[16] M. Randevik and P. Olson, "SecArchUnit: Extending ArchUnit to support validation of security architectural constraints," M.S. thesis, Chalmers University of Technology and University of Gothenburg, 2020. [Online]. Available: https://odr.chalmers.se/server/api/core/bitstreams/4851b5ce-4247-4200-8f56-67fbfa7015ab/content.

[17] S. Miranda, E. Rodrigues Jr., M. T. Valente, and R. Terra, "Architecture conformance checking in dynamically typed languages," *Journal of Object Technology*, vol. 15, no. 3, 1:1–1:34, 2016. DOI: 10.5381/jot.2016.15.3.a1. [Online]. Available: https://www.jot.fm/issues/issue_2016_03/article1.pdf.

[18] A. Ivanova, G. Kolev, F. Sapundzhi, T. Bakardjieva, and S. Georgiev, "Integration of large language models in layered software systems: A clean architecture and CQRS case study," *Engineering Proceedings*, vol. 154, no. 1, p. 23, 2026. DOI: 10.3390/engproc2026154023. [Online]. Available: https://www.mdpi.com/2673-4591/154/1/23.

[19] S. Souza, A. Santana, E. Figueiredo, I. Muzetti, J. E. Montandon, and L. Briand, *Beyond strict rules: Assessing the effectiveness of large language models for code smell detection*, 2026. arXiv: 2601.09873. [Online]. Available: https://arxiv.org/pdf/2601.09873.

[20] R. Su, A. Bakhtin, N. Ahmad, M. Esposito, V. Lenarduzzi, and D. Taibi, *Evaluating large language models for detecting architectural decision violations*, 2026. arXiv: 2602.07609. [Online]. Available: https://arxiv.org/html/2602.07609v1.

[21] A. Lucena, E. L. G. Alves, and J. Brunet, "Language models as architectural gatekeepers: Automating conformance checking from natural language," in *Proceedings of the 39th Brazilian Symposium on Software Engineering (SBES)*, Brazilian Computer Society (SBC), 2025, pp. 713–719. DOI: 10.5753/sbes.2025.11100.